

Programación Lógica - PROLOG

El lenguaje de programación PROLOG ("PROgrammation en LOGique") fue creado por Alain Colmerauer y sus colaboradores alrededor de 1970 en la Universidad de Marseille-Aix¹, si bien uno de los principales protagonistas de su desarrollo y promoción fue Robert Kowalski² de la Universidad de Edimburgh. Las investigaciones de Kowalski proporcionaron el marco teórico, mientras que los trabajos de Colmerauer dieron origen al actual lenguaje de programación, construyendo el primer intérprete Prolog. David Warren³, de la Universidad de Edimburgh, desarrolló el primer compilador de Prolog (WAM – "Warren Abstract Machine"). Se pretendía usar la lógica formal como base para un lenguaje de programación, es decir, era un primer intento de diseñar un lenguaje de programación que posibilitara al programador especificar sus problemas en lógica. Lo que lo diferencia de los demás es el énfasis sobre la especificación del problema. Es un lenguaje para el procesamiento de información simbólica. PROLOG es una realización aproximada del modelo de computación de Programación Lógica sobre una máquina secuencial. Desde luego, no es la única realización posible, pero sí es la mejor elección práctica, ya que equilibra por un lado la preservación de las propiedades del modelo abstracto de Programación Lógica y por el otro lado consigue que la implementación sea eficiente.

El lenguaje PROLOG juega un importante papel dentro de la Inteligencia Artificial, y se propuso como el lenguaje nativo de las máquinas de la quinta generación ("Fifth Generation Kernel Language", FGKL) que quería que fueran Sistemas de Procesamiento de Conocimiento. La expansión y el uso de este lenguaje propició la aparición de la normalización del lenguaje Prolog con la norma ISO (propuesta de junio de 1993).

PROLOG es un lenguaje de programación para ordenadores que se basa en el lenguaje de la Lógica de Primer Orden y que se utiliza para resolver problemas en los que entran en juego objetos y relaciones entre ellos. Por ejemplo, cuando decimos "Jorge tiene una moto", estamos expresando una relación entre un objeto (Jorge) y otro objeto en particular (una moto). Más aún, estas relaciones tienen un orden específico (Jorge posee la moto y no al contrario). Por otra parte, cuando realizamos una pregunta (¿Tiene Jorge una moto?) lo que estamos haciendo es indagando acerca de una relación. Además, también solemos usar reglas para describir relaciones: "dos personas son hermanas si ambas son mujeres y tienen los mismos padres". Como veremos más adelante, esto es lo que hacemos en Prolog. Una de las ventajas de la programación lógica es que se especifica qué se tiene que hacer (programación declarativa), y no cómo se debe hacer (programación imperativa). A pesar de esto, Prolog incluye algunos predicados predefinidos metalógicos, ajenos al ámbito de la Lógica de Primer Orden, (var, nonvar, ==, ...), otros extra-lógicos, que tienen un efecto lateral, (write, get, ...) y un tercer grupo que nos sirven para expresar información de control de cómo realizar alguna tarea (el corte, ...).

Por tanto, Prolog ofrece un sistema de programación práctico que tiene algunas de las ventajas de claridad y declaratividad que ofrecería un lenguaje de programación lógica y, al mismo tiempo, nos permite un cierto control y operatividad.

¹ A. Colmerauer. Les Systèmes-Q, ou un formalisme pour analyser et synthétiser des phrases sur ordinateur. Internal Report 43, Computer Science Dpt., Université de Montreal, septembre, 1970.

A. Colmerauer, H. Kanoui, P. Roussel y R. Pasero. Un Systeme de Communication Homme-Machine en Français, Groupe de Recherche en Intelligence Artificielle, Université d'Aix-Marseille, 1973.

² R. Kowalski. Predicate Logic as a Programming Language. En Proc. IFIP, Amsterdam, 1974.

³ D. Warren. The runtime environment for a prolog compiler using a copy algorithm. Technical Report 83/052, SUNY and Stone Brook, New York, 1983.

Contenidos

1. Conceptos Básicos (hechos-consultas-reglas)
2. Unificación
3. Procedimientos
4. Recursividad
5. Estructuras de Control
6. Listas

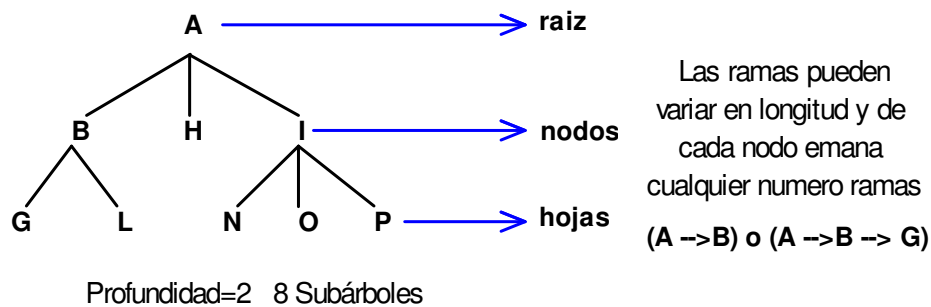
PROLOG: Programación Lógica

La programación lógica consiste en 3 actividades

- Declaración de Hechos
- Establecimiento de reglas deductoras basadas en los hechos
- Consultas (preguntas)

Conceptos Básicos

1. TREES: base del proceso realizado en y por PROLOG (syntaxis)



Un árbol permite la definición de una “familia de relaciones”: A es el padre de B, H e I; B es hermano de H e I, etc.

El árbol es la única estructura de datos utilizada en PROLOG. Y usa una notación llana:
A(B(G,L),H,I(N,O,P))

HECHOS

PROLOG consiste esencialmente en un mecanismo automático de prueba. Este mecanismo efectúa un razonamiento basado en el conocimiento que el programador brinda al sistema, y sobre el que posteriormente se podrán realizar consultas.

El conocimiento se representa mediante hechos y reglas.

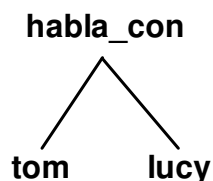
El enunciado “Tom habla con Lucy” puede ser descompuesto en dos *objetos* identificados por los nombres “Tom” y “Lucy”, y una *relación* entre ellos designada como “habla con”. En lógica Tom y Lucy son constantes. La relación que se establece entre objetos se considera predicado.

predicado(argumento₁, argumento₂,...,argumento_n)

habla_con(tom,lucy).

Los predicados tienen un valor de verdad, serán verdaderos o falsos.

A nivel sintáctico el enunciado habla_con(tom,lucy) es considerado un término. A nivel semántico es un hecho.



PROPIEDADES Y RELACIONES

De acuerdo al significado de los hechos.

Propiedad: es_bueno(juan). Relación: habla_con(tom,lucy).

Cuando un hecho tiene un solo argumento, por lo general es una propiedad. Un hecho representa una relación cuando tienen más de un argumento. Es posible definir hechos sin argumentos: lindo_dia.

Los hechos son la parte integral de un programa en PROLOG. Los hechos que pertenecen a un programa se consideran verdaderos. Los hechos que no pertenecen a un programa se consideran falsos. El conjunto de hechos de un programa se denomina “base de hechos” o “base de conocimiento”.

CONSULTAS

Son preguntas al sistema sobre el conocimiento(hechos) almacenado.

1. Consultas elementales

Tiene la misma sintaxis que un hecho, para distinguir las consultas van precedidas de “-“(signo menos) que lo da el sistema luego del prompt “?”

?- consulta.

?- habla_con(tom,lucy). Se pregunta si tom habla con lucy.

PROLOG busca en la base de hechos si esta información ha sido introducida previamente. Si existe PROLOG contesta YES (verdadero) o NO (falso)

Para que conteste verdadero:

1. el nombre de la consulta debe ser exactamente igual al nombre de la relación
2. los correspondientes argumentos son idénticos.

Dada la siguiente base de hechos:

hermano(paul,caty).
hermano(paul,lucy).
hermano(john,luke).
hermano(luke,michael).
hermano(michael,edith).
hermano(paul,martine).

El orden en que los hechos son declarados no afecta el valor de verdad del resultado. PROLOG atraviesa la base hasta encontrar la respuesta.

Ejemplo

?- hermano(michael,lucy).

PROLOG examina la base de hechos. Cada hecho en el orden en que fueron declarados. Cuando encuentra que una consulta concuerda con un hecho, NO se detiene, continua buscando hasta el final. Si el hecho está varias veces en la base de hechos, será encontrado todas las veces. Es decir,

PROLOG encuentra todas las posibles soluciones a un problema. Hay métodos para detener la búsqueda.

PROLOG es un conjunto de hechos que pueden ser considerados como un conjunto de axiomas (considerados verdaderos a priori). Una consulta es entonces un teorema, que se desea demostrar – probar a partir de los axiomas. PROLOG es el mecanismo que permite al sistema construir la prueba y determinar si el teorema es deducible de los axiomas.

Las consultas se denominan objetivos: $?- q()$.

Una consulta que no puede probarse, no significa necesariamente que es falsa, quizás simplemente no exista en la base de hechos($?- hermano(ana,juan).$).

2. Consultas con Variables

Para preguntar: ¿quién es el hermano de paul?

$?- hermano(paul,X)$. X: variable, va con mayúscula.

Traerá todos los valores para X en los cuales la relación es “hermano” tenga en el primer argumento el valor paul.

El concepto de variable es diferente al de los lenguajes convencionales, no representa una posición en memoria, sino una entidad específica.

La respuesta de la consulta planteada, no es única, según la base de hechos. Paul es hermano de caty, lucy y martine.

Al comienzo de la evaluación de la consulta, X no es limitada (ligada) a ningún valor. X es libre. La tarea de PROLOG es encontrar en la base de hechos todos los posibles valores que limiten a X para responder a la consulta. Para encontrar todos los valores de X que hagan a la consulta $hermano(paul,X)$ verdadero, PROLOG procede de la siguiente manera:

0. Considera la primer ocurrencia del hecho hermano en la base de hechos. Lo cual llamamos “hecho actual”.
1. Examina todas las ocurrencias de hermano en la base de hechos, comenzando desde el primero hasta que encuentre un hecho que **unifique** con la constante que aparece en la consulta (paul)
2. Entonces, liga, limita X al valor que encuentra en el correspondiente argumento en el hecho. Este hecho se convierte en el hecho actual.
3. Retorna (muestra) el resultado.
4. Entonces, libera X. X es ahora libre para ser limitada por otros valores.
5. Mientras no haya explorado todas las ocurrencias del hecho hermano en la base de hechos, procede al siguiente hecho y vuelve al paso 1. En otro caso la evaluación finaliza.

Las variables que aparecen en las consultas se dicen que están “existencialmente cuantificadas”. Esto significa que $hermano(paul,X)$ es verdadero si existe un valor para X tal que $hermano(paul,<valor>)$ es verdadero. Cuando hay varias soluciones a una consulta, la primer respuesta es retornada y las siguientes se obtienen tipeando “;” (punto y coma)

X = caty;

X = lucy;

X = martine;

yes

3. Conjunción de consultas elementales

Para preguntar: ¿son caty y lucy hermanas de paul?

Esta consulta significa a nivel de PROLOG: existe un hecho tal que determine que paul es el hermano de caty y existe otro hecho tal que determine que paul es hermano de lucy.

?- hermano(paul,caty), hermano(paul,lucy).

PROLOG retorna yes a la consulta si y solo si ambos hechos existen en la base de hechos.

Para preguntar: ¿caty y martine tienen el mismo hermano?

Esta consulta requiere usar una variable. Para expresar este problema en PROLOG debemos razonar en la siguiente manera:

1. Obtener en X los hermanos de caty, si existe alguno.
2. La respuesta será verdadera si también existe un hecho que establezca que X es hermano de martine.

?- hermano(X,caty), hermano(X,martine).

- La variable X es usada dos veces en la consulta para designar el mismo individuo.

1. PROLOG primero intenta probar la primer parte de la consulta: encontrar un X que sea hermano de caty. Si encuentra algún valor, entonces la segunda ocurrencia de X en la consulta es inmediata y automáticamente limitada a ese valor:

?- hermano(paul,caty), hermano(paul,martine).

2. Luego el sistema busca en la base de hechos si existe un hecho que coincida con la segunda parte de la consulta. (X ya no es libre)

- Cuando la primer parte de la consulta encuentra una solución, tiene el efecto de ligar X a paul, es necesario desde ahí demostrar que hermano(paul,martine) es verdadero. Si ese hecho existe la consulta tiene solución. Una consulta será verdadera si y solo si existe una solución que satisfice cada parte de la consulta.
- Mecanismo de ejecución de PROLOG: si PROLOG encuentra que un valor X ligado en la primer parte de la consulta no satisface la segunda parte de la consulta, reconsidera este valor liberando X a la primer parte de la consulta y buscando una nueva solución que satisfaga toda la consulta. Si no encuentra una solución, la respuesta es NO.
- El acto de reconsiderar una elección previa es llamado **“backtracking”**. El backtracking es implementado en PROLOG por pilas. Lo importante es que si existen varias posibles respuestas para una consulta, PROLOG con el backtracking encuentra a todas.

Ejemplo

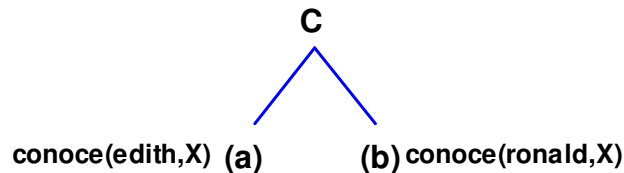
Base de Hechos

```
conoce(john,luke).  
conoce(edith,colette).  
conoce(ronald,colette).  
conoce(james,snoopy).  
conoce(edith,james).  
conoce(ronald,archie).  
conoce(james,archie).
```

conoce(edith,archie).

Pregunta: ¿Qué personas son conocidas por edith y ronald?
?- conoce(edith,X), conoce(ronald,X).

La consulta a probar puede representarse por un pequeño diagrama de árbol, en el que cada rama designa un subproblema.

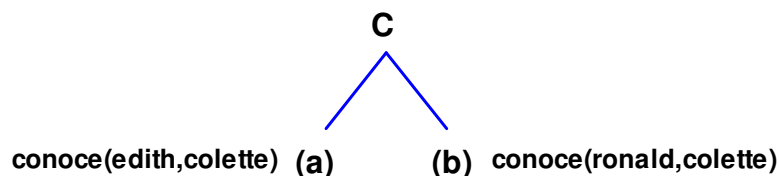


Aquí hay dos subproblemas (a) y (b). PROLOG siempre comienza buscando una solución con el subárbol más a la izquierda. En el subárbol (a). El proceso de **resolución** completo es:

1.

(a): X es ligada a colette

(b): El valor de X automáticamente es asignado a la segunda parte de la consulta, conoce(ronald,colette) es verdadero, por lo tanto la primer respuesta es colette.



2.X es liberada del valor colette. La búsqueda continúa:

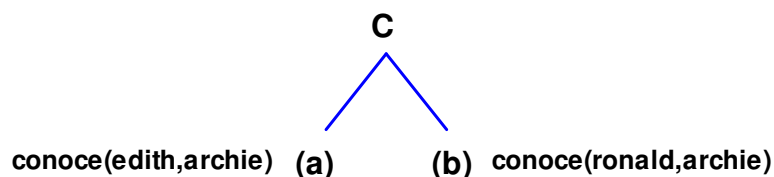
(a): X es ligada a james

(b): PROLOG no encuentra un hecho que pruebe conoce(ronald,james). El segundo intento de probar, falla o fracasa.

3.X es liberada del valor james. La búsqueda continúa:

(a): X es ligada a archie

(b): PROLOG encuentra un hecho que pruebe conoce(ronald,james). El tercer intento de probar, es la segunda solución.



4.X es liberada del valor archie, pero no hay otros valores para la parte (a) de la consulta. La búsqueda termina luego que la base de hechos ha sido completamente atravesada.

REGLAS

Permiten expresar el conocimiento de manera general.

Consideremos la siguiente relación: hijo(x,y). Establece de manera general que x es el hijo de y.

Base de Hechos

hijo(john,jack).

hijo(caty,marcel).
hijo(edith,sylvia).
hijo(philip,edith).
hijo(sylvia,julie).
hijo(sylvia,john).
hijo(max,caty).

Para agregar la relación abuelo(x,y), que establece que x es el abuelo de y, se puede hacer de dos maneras:

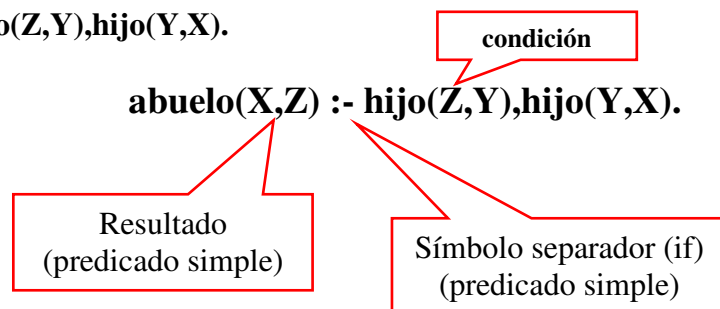
- 1) crear un nuevo conjunto de hechos, por ej. abuelo(sylvia,jack), etc.
- 2) Definir una regla.

Construir una regla

En este caso existe una relación entre hijo y abuelo, ya que:

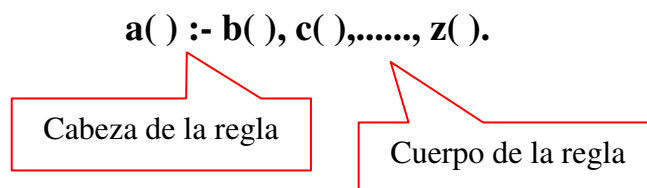
Z		y		x
Hijo	→	padre	→	abuelo
Hijo(Z,Y)		y		hijo(Y,X)

abuelo(X,Z) :- hijo(Z,Y),hijo(Y,X).



Forma general de una regla

a() :- b(), c(),....., z(). Significa que a es verdadera si b,c,...,z son verdaderas.



- Sintácticamente una regla se denomina “**cláusula**”. El cuerpo son condiciones o restricciones. Las condiciones pueden hacer referencias a hechos o a otras reglas.
- Regla: manera mas eficiente y general de describir la información:
 - Es general por que contiene variables
 - Es eficiente en términos de memoria
 - Se convierte en “super hecho”

Consulta: **?- abuelo(X,max).**

Es decir, queremos conocer quien es el abuelo de max, a partir de los hechos de la base. La consulta no puede resolverse directamente consultando los hechos, se debe usar la regla abuelo, ya que no existen hechos abuelo.

PROLOG procede como sigue:

- (1) Inmediatamente trata de unificar la consulta con la cabeza de la regla abuelo(,). La unificación es posible y Z es ligada a max. Esto implica que todas las ocurrencias de Z en la regla son ligadas a max. La regla queda: abuelo(X,max):-hijo(max,Y),hijo(Y,X).
- (2) El sistema ahora trata de probar la parte de la condición de la regla, paso a paso, considerando los subproblemas en el orden en que aparecen. Primero trata de encontrar un hecho (u otra regla) que coincida hijo(max,Y) y encuentra Y=caty. Todas las ocurrencias de Y son reemplazadas por caty. abuelo(X,max):-hijo(max,caty),hijo(caty,X).
- (3) El sistema intenta probar la última parte de la condición hijo(caty,X). Encuentra X=marcel.
- (4) El valor de X es propagado automáticamente a la cabeza de la regla, donde X hasta ahora permanece libre. El valor de X es mostrado.

Luego, desliga X, y el sistema busca otras soluciones. El sistema procede de la misma manera atravesando toda la base de hechos.

➤ Podríamos poner la consulta inversa, **?- abuelo(marcel,X)**. Que significa preguntar quienes son los nietos de marcel.

➤ Una característica interesante de las reglas de PROLOG es que es posible instanciar o dejar libre cualquier argumento de la cabeza. Esto permite la resolución de diferentes clases de consultas con la misma regla.

Para expresar la relación **padre(X,Y)**. Donde X es el padre de Y, es posible usar la relación hijo. **padre(X,Y):-hijo(Y,X)**.

Si además conocemos el sexo (masc o fem) de los individuos:

masc(john).
masc(jack).
masc(marcel).
masc(philip).
masc(max).
fem(edith).
fem(silvia).
fem(martine).
fem(julie).
fem(caty).

Podemos definir las relaciones:

padre(X,Y):-hijo(Y,X),masc(X).
madre(X,Y):-hijo(Y,X),fem(X).
abuelo(X,Y) :- hijo(Y,Z),hijo(Z,X),masc(X).
abuela(X,Y) :- hijo(Y,Z),hijo(Z,X),fem(X).

La regla padre puede expresarse como:

Para todo X e Y, X es el padre de Y si:

- (1) Y es el hijo de X
- (2) X es de sexo masculino

La regla abuelo puede expresarse como:

Para todo X e Y, X es el abuelo de Y si:

- (1) existe un Z tal que Y es el hijo de Z
- (2) Z es el hijo de X
- (3) X es de sexo masculino

Reglas sin condiciones

Hay una clase de reglas que no tienen un componente condicional. Estas reglas pueden considerarse como hechos que tienen argumentos variables.

Teníamos en la base de hechos la relación conoce(X,Y) que establece que x conoce a Y. Pero la siguiente regla: conoce(X,X) no contiene condiciones y expresa “para todo X, X se conoce así mismo”(propiedad reflexiva). Esta regla debe ser usada con cautela. Si la base de hechos no solo describe humanos, sino también describe cosas se debe hacer:

conoce(X,X):-persona(X).

Ejemplo

igual(X,X) // X es igual a si mismo

Reglas y lógica

Una regla como un hecho es un axioma en lógica. La regla abuelo(X,Z) :- hijo(Z,Y),hijo(Y,X) implica:

Si Z es el hijo de Y e Y es el hijo de X, entonces X es el abuelo de Z. Las variables de la cabeza están universalmente cuantificadas, la regla es verdadera o falsa para diferentes valores de X, Z e Y.

Notación Lógica

$$\forall X,Z \exists Y, \text{hijo}(Z,Y) \wedge \text{hijo}(Y,X) \Rightarrow \text{abuelo}(X,Z)$$

Sintaxis de PROLOG – Mecanismo de Unificación

PROLOG manipula:

- Objetos elementales: átomos, números, variables
- Objetos estructurados: términos
- Cláusulas: hechos y reglas

Números: positivos y negativos, reales y enteros (12, -13, 0.15, -3.14)

Átomos: un átomo es un identificador, el nombre de constante, un predicado o un operador. (john, +)

Variables: carácter en mayúscula seguido de más caracteres o subrayado (X, X1)

Términos: los números, constantes y variables en un programa son llamados términos. Un árbol es un término, una estructura compuesta de números, constantes y variables. (padre(X,Y) – conoce(juan,Y))

La raíz del árbol es llamado cabeza del término: a(b(c,d1),Z,Y) a es la raíz o cabeza. Tiene 3 términos b(c,d1), Z e Y.

La aridad de a es 3, la aridad del subtérmino b es 2.

Un término se dice que está completamente instanciado si no contiene variables. g(1,f(pepe))

Cláusulas: a nivel sintáctico una regla se denomina cláusula

Forma general: a(...) :- b(...),...,z(...).

Donde a(...) se llama cabeza del literal de la cláusula y b(...),...,z(...) constituyen el cuerpo de la cláusula (condición), también son literales.

a(...), b(...), z(...) son términos. Sus argumentos también son términos y pueden ser simples o complejos. b(...),...,z(...) pueden ser hechos u otras reglas.

- Cuando una cláusula a es llamada, tiene lugar las siguientes acciones:
 - (1) Unificación de la consulta (a) con la cabeza de la cláusula;
 - (2) Intenta probar primero b, y luego todas las siguientes condiciones en el orden en que están escritas.

Las variables contenidas en una cláusula son locales a esa cláusula.

a(X):-b(X).

c(X):- f(X),g(X).

La variable X de la primer cláusula es independiente de la X de la segunda cláusula. Durante la ejecución PROLOG crea nuevas variables para evitar conflicto entre variables con el mismo nombre.

Substituciones

La substitución es la operación básica del mecanismo de unificación. Sustituir una variable X por un término, es reemplazar todas las ocurrencias de esa variable por el término (y no lo opuesto). Se cae en un bucle infinito si el término del reemplazo contiene ocurrencias de la variable a reemplazar.

$X \rightarrow f(...,X,...)$.

Si se presenta este caso, es necesario renombrar la X en f. Si no se hace esto, podemos substituir X en un numero infinito de veces con f(..,X,...), lo cual da: f(...f(...f(...f..)))

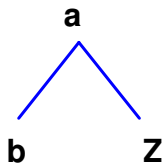
Consideremos la substitución S(X,t), que reemplaza todas las ocurrencias de X por el término t en otro término. Por ejemplo:

$S(X, a(b, Z))$ aplicado al término $V = b(X, g(Y, X))$

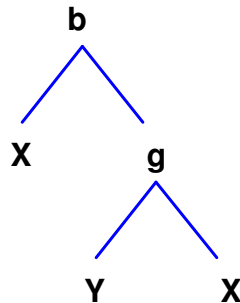
Resulta:

$S(V) = b(a(b, Z), g(Y, a(b, Z)))$

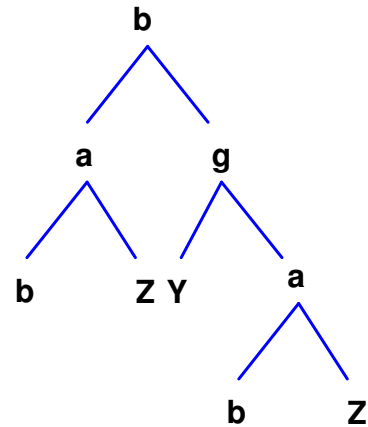
S reemplaza X por:



El término U es:



S(U) es:



La sustitución $S(X, a(b, Z))$ aplicada a $T1 = f(Y, Z)$ deja sin cambiar a $T1$, porque $T1$ no contiene ninguna ocurrencia de X .

Unificación de Términos

Dos términos T y U son unificables, si y solo si, existe uno o varias substituciones que, aplicados simultáneamente a T y a U , haga a estos dos términos "idénticos". Si no existe al conjunto de substituciones, los dos términos no son unificables.

Ejemplo:

Términos	Conjunto de substituciones S a aplicar a T y U	Substituciones
$T = p(X, f(a, b), g(Z, F))$	$S = \{S1(X, T(e)), S2(W, f(a, b)), S3(Z, 3), S4(F, C)\}$	$T = p(t(e), f(a, b), g(3, C))$
$U = p(t(e), W, g(3, C))$		$U = p(t(e), f(a, b), g(3, C))$

La **unificación** es la operación fundamental en PROLOG. Permite la manipulación y transferencia de datos, como la expresión de condiciones sobre los datos, a través del uso de varias ocurrencias de la misma variable en una cláusula. La unificación también permite determinar con la ayuda de la substitución, si dos árboles (términos) son idénticos.

La unificación permite acceder a distintos subcomponentes de un término a través del uso de variables que puedan ser unificadas con la parte de un término. Por ejemplo para extraer $a(b, c)$ del término $t(c, a(b, c), Y)$, simplemente se unifica este término por el término $t(_, X, _)$ y X es ligada a $a(b, c)$. X puede ser usada en el resto de la cláusula para transportar el término que ha sido ligado.

Un término T es una instancia del término U si existe una substitución (o grupo de S) al que: $T = S(U)$. T es el resultado de aplicar S a U .

Operaciones Aritméticas

Operadores: +, -, *, /, mod

Para limitar una variable al valor de una expresión aritmética es necesario usar el operado “is”

?- X is 3 - 2.

X=1

?- X is (3 + 2) * 10.

X=50

También es posible usar variables a la derecha del operador is.

incremento(X,Y) :- Y is X + 1. // regla

?- incremento(5,Y).

Y=6

promedio(X1,X2,X3,X4,P) :- P is (X1 + X2 + X3 + X3 +X4) / 4. // regla

?- promedio(5,7,10,2,X).

X=12

Operaciones Lógicas

Operadores: >, <, >=, <=, \=

?- 120 >= 20.

yes

Cuando se usa una variable en una expresión lógica, debe estar previamente instanciada. Sino la comparación no es valida. (comparar 2 variables no tiene sentido)

Los operadores lógicos trabajan con números, hay 3 operadores lógicos que trabajan en términos de:

$X = Y$ es verdadero si X unifica con Y

$X = = Y$ es verdadero si X e Y tienen exactamente la misma estructura, argumento por argumento.

$X \backslash = Y$ es verdadero si X e Y no tienen la misma estructura.

- “Exactamente la misma estructura” significa que el árbol representado por X reviste exactamente el árbol representado por Y y viceversa.
- Para un simple igualdad (=) es suficiente que X e Y sean unificables, esto es, que la variable X puede ser unificada con un subárbol en Y.
- En el caso de estrictamente igual (==), cuando hay una constante en X, la misma constante debe existir en Y. Donde quiera que exista una variable en X, debe existir una variable en Y.

PROCEDIMIENTOS

En PROLOG se puede definir varias reglas con la misma cabeza. Esta técnica permite establecer independientemente más posibles formas de probar un objetivo, cuando existen varios caminos.

Problema: definir una regla $\text{max}(X,Y,R)$ que ligue a R el mayor de 2 números X e Y , si son iguales X es el mayor:

$R = X$ if $X \geq Y$

$R = Y$ if $Y > X$

$\text{max}(X,Y,X) :- X \geq Y.$

$\text{max}(X,Y,Y) :- Y > X.$

Mutuamente excluyentes
(disyunción-or)

?- $\text{max}(12,4,X).$

$X=12$

En la parte de las condiciones puede haber hechos, reglas, predicados predefinidos.

Un *procedimiento* agrupa un conjunto de cláusulas que tienen el mismo término (el mismo nombre, la misma cantidad de argumentos) como cabeza de cláusula.

$a(X) :- b(X).$

$a(X) :- c(X), e(X).$

$a(X) :- f(X).$

Este procedimiento contiene 3 cláusulas con la cabeza $a(X)$. Cada cláusula del procedimiento expresa una forma de resolver $a(X)$. El ejemplo podría explicarse: $a(X)$ es verdadero si $b(X)$ es verdadero o si $c(X)$ y $e(X)$ son verdaderos o si $f(X)$ es verdadero.

➤ Entre las cláusulas del procedimiento hay un OR implícito. Esto es absolutamente diferente a los lenguajes imperativos. El AND es explícito.

➤ En un procedimiento las cláusulas son mutuamente excluyentes. Esto no es una propiedad inherente de las cláusulas de un procedimiento. Cada cláusula es meramente una forma de definir una propiedad, una relación, una situación, etc. Pero dos o más cláusulas pueden no ser excluyentes.

Ejemplo: un ave se define como

$\text{ave}(X) :- \text{vuela}(X).$

$\text{ave}(X) :- \text{tiene_plumas}(X).$

$\text{ave}(X) :- \text{tiene_pico}(X).$

Es posible agrupar condiciones dos cláusulas de un procedimiento en una simple cláusula.

Ejemplo:

$a(X) :- b(X,Y),c(X),d(Y,Z).$

$a(X) :- b(X,Y),g(X,Z),d(Y,Z).$



$a(X) :- b(X,Y),(c(X);g(X,Z)),d(Y,Z).$

Orden de ejecución de las cláusulas en un procedimiento

Teóricamente, el orden de las cláusulas en un procedimiento no es importante. Esto no es del todo verdad, por razones de eficiencia. Para esto se debe influir en el control interno de PROLOG y para ello hay que prestar atención en el orden de las cláusulas.

➤ Desde el punto de vista de ejecución, cuando un procedimiento es llamado, PROLOG ejecuta las cláusulas una después de la otra, comenzando con la primera. Si PROLOG falla al probar una cláusula pasa a la siguiente cláusula y continua de esta manera hasta alcanzar la última cláusula. Si todos los intentos de probar una cláusula fallan, el resultado es falso. Para que el resultado sea verdadero, al menos una cláusula del procedimiento debe ser verdadero.

➤ A nivel de procedimiento cuando PROLOG analiza un programa crea un puntero (puntero de entrada) para cada procedimiento. Cada cláusula de cada procedimiento es numerada de acuerdo a su orden de aparición. Esa numeración es usada durante la ejecución de un programa las cláusulas son ejecutadas en el orden en que son numeradas dentro de cada procedimiento. Ejemplo:

c :- a,b.

c :- d.

c :- e,a.

PROLOG produce la siguiente numeración para el procedimiento c:

(1) c :- a,b.

(2) c :- d.

(3) c :- e,a.

El puntero de entrada es la cláusula (1). Durante la ejecución del programa, una llamada a c comienza la ejecución del procedimiento en el orden determinado (1-2-3).

Suponer que a, b, d y e son procedimientos definidos en el programa:

- (1) La llamada a c se inicia intentando probar la primer cláusula que intentara probar primero a y después b (substitución-unificación)
- (2) En el caso que (1) falle, o no (para encontrar otra solución) PROLOG intenta probar la (2) cláusula, que requiere probar d. El proceso continua.
- (3) Luego de examinar la última cláusula del procedimiento c, falla si ninguna cláusula del procedimiento pudo ser probada.

Procedimientos y programación clásica

Los procedimientos PROLOG se pueden comparar con los procedimientos de los lenguajes imperativos. Como en los lenguajes imperativos, un procedimiento en PROLOG da una forma completa de solucionar un problema. Pero puede verse mas declarativo ya que se provee de una completa definición de una relación o propiedad. Consecuentemente dos procedimientos en PROLOG no pueden tener la misma cabeza.

Un procedimiento clásico usa la estructura if..then..else... El procedimiento max en pseudo-pascal sería

```
procedure max(X,Y,R)
begin
  if X >= Y then R := X
  else   if X < Y then R := Y
end
```

RECURSIVIDAD

Las soluciones recursivas implican el siguiente esquema:

- Una condición inicial que inicia el proceso
- Un plan de desarrollo que contienen la llamada recursiva
- Una condición de fin, que generalmente es expresada en otro procedimiento.

La siguiente cláusula tiene una llamada recursiva, ya que en su cuerpo se llama a sí mismo:

a(X) :- b(X,Y), a(Y).

La cláusula a se llama a sí misma, se ejecuta indefinidamente y produce un bucle infinito. Una simple solución a este problema es agregar una condición de fin, expresado por otra cláusula con la cabeza a. Esta cláusula, llamada condición de fin, debe ser examinada antes que cada nueva llamada recursiva, un requerimiento que implica que debe ser el primer llamado al procedimiento. También existen otras técnicas.

Ejemplo: se quiere obtener la siguiente secuencia de 10 enteros que van del 2 en 2. 0-2-4-6-8-10-12-14-16-18

Se usa el predicado numero(X,E), donde X designa el elemento de la secuencia y E el número de nivel.

number(X,E) :- write(X), E1 is E + 1, X1 is X + 2, number(X1,E1).

Condición de fin: number(X,10).

Esquema:

number(X,10).

number(X,E) :- write(X),
 E1 is E + 1,
 X1 is X + 2,
 number(X1,E1).

Condición de fin: number(X,10).

Consultas:

?- number(0,0).

?- number(14,0).

Este procedimiento recursivo se compone de 2 cláusulas. La condición de fin es evaluada durante cada nueva llamada a number, debe ser la primer cláusula del procedimiento,

La recursividad puede ser expresada de un modo menos directo que el anterior. Puede ocurrir en el intermedio de dos o más procedimientos:

a(1).

a(X) :- b(X).

b(X) :- X1 is X - 2, a(X1).

El procedimiento a llama a b, que nuevamente llama a a. (recursividad indirecta)

La recursividad en una cláusula puede tener dos formas:

- Recursividad a derecha

$a(X) :- b(X,Y), a(X).$

La llamada a $a()$ esta al final de la cláusula. Es la forma mas utilizada y simple de control.

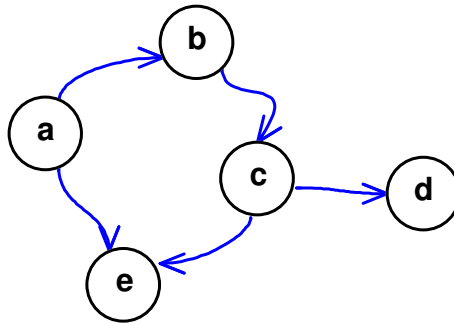
➤ Recursividad a izquierda

$a(X) :- a(X), b(Y,Z).$

La llamada a $a()$ esta al principio de la cláusula. Esta forma debe ser usada con cuidado porque puede causar bucles infinitos.

➤ Ejemplo

Determinar la existencia de uno o varios caminos entre los nodos de un grafo.



Un arco expresa la relación entre dos nodos $\text{arco}(X,Y)$ establece la existencia de un arco que va desde X a Y, donde X e Y son nodos del grafo.

Base de Hechos

$\text{arco}(a,b).$
 $\text{arco}(b,c).$
 $\text{arco}(c,d).$
 $\text{arco}(c,e).$
 $\text{arco}(a,e).$

Consulta ¿hay camino de a a d?

Debemos definir el predicado $\text{camino}(X,Y)$, para evaluar si es verdadero que existe camino entre X e Y. El problema debe ser descompuesto en dos casos, expresado por dos cláusulas para el procedimiento camino. La primer cláusula considera el caso en el cual el camino es elemental (directo): hay camino entre dos puntos si existe un arco que los une. La segunda cláusula considera el caso en el que el camino no es elemental (indirecto): hay camino entre X e Y si existe un Z para el cual hay arco entre X y Z y existe camino entre Z e Y.

$\text{camino}(X,Y) :- \text{arco}(X,Y).$ // condición de fin

$\text{camino}(X,Y) :- \text{arco}(X,Z), \text{camino}(Z,Y).$ // caso general

➤ Ejecución de la consulta: **?- camino(a,d).**

PROLOG numera las cláusulas del procedimiento camino de acuerdo al orden en que están escritas:

(1) $\text{camino}(X,Y) :- \text{arco}(X,Y).$

(2) $\text{camino}(X,Y) :- \text{arco}(X,Z), \text{camino}(Z,Y).$

La ejecución del proceso es como sigue:

► Nivel 1

Cláusula (1): $\text{camino}(X,Y) :- \text{arco}(X,Y).$

Unifica con la llamada: camino(a,d) :- arco(a,d). FALLA

Cláusula (2): camino(X,Y) :- arco(X,Z), camino(Z,Y).

Unifica con la llamada: camino(a,d) :- arco(a,Z), camino(Z,d).

El hecho arco(a,b) permite la unificación : Y=b y debe ser probado el objetivo camino(b,d)

► Nivel 2

Cláusula (1): camino(X,Y) :- arco(X,Y).

Unifica con la llamada: camino(b,d) :- arco(b,d). FALLA

Cláusula (2): camino(X,Y) :- arco(X,Z), camino(Z,Y).

Unifica con la llamada: camino(b,d) :- arco(b,Z), camino(Z,d).

El hecho arco(b,c) permite la unificación : Y=c y debe ser probado el objetivo camino(c,d)

► Nivel 3

Cláusula (1): camino(X,Y) :- arco(X,Y).

Unifica con la llamada: camino(c,d) :- arco(b,d). EXITO

Cuando la llamada del nivel 3 es probada, en la (1) del procedimiento camino (fin de recursión), llamada 2 es probada, no hay mas condiciones a la izquierda que probar y por la misma razón la llamada 1 es probada.

➤ La serie de llamadas recursivas finaliza cuando no hay mas literales en las llamadas precedentes aguardando para ser ejecutadas, se deduce que la consulta completa ha sido contestada. Cuando hay otras soluciones PROLOG busca todas ellas. Este sería el caso por ejemplo, si se agrega el hecho arco(b,d). Habría dos caminos de a a d. Puede producirse un bucle infinito si el grafo es cíclico.

➤ El programa camino no da el camino entre X e Y. Solo indica si existe o no camino entre X e Y. Este programa puede ser modificado para que de también el camino, para eso debe usar “listas”

Si el programa camino fuera:

camino(X,Y) :- arco(X,Y). // condición de fin

camino(X,Y) :- camino(X,Z), arco(Z,Y). // caso general

es lógicamente correcto, introduce un bucle infinito por que Z esta libre hasta la ejecución de arco(Z,Y).

➤ Si PROLOG usara una estrategia de búsqueda breadth-first (primero en anchura), no habría bucle infinito pero utiliza una estrategia de búsqueda depth-first(primero en profundidad)

➤ Cuando se escribe una cláusula recursiva se deben considerar dos factores:

(2) la formulación lógica del problema

(3) la manera en que el programa será ejecutado

Hay otras formas de usar el programa camino, dependiendo de si los argumentos son libres o instanciados:

?- camino(X,d).

PROLOG da el conjunto de nodos X del grafo desde los cuales existe camino a d. PROLOG da todas las soluciones.

?- camino(a,X).

PROLOG da todos los nodos X que son accesibles desde a.

Problema: Calcular la longitud del camino

Queremos conocer la longitud de un camino, número de arcos que lo atraviesan desde el nodo inicial hasta el nodo final. Para escribir el programa debemos razonar de la siguiente manera:

1. Si el camino desde un punto a otro es elemental (directo), su longitud es 1. Debemos agregar un tercer argumento a la cláusula camino:

camino(X,Y,L).

Donde L es la longitud del camino X-Y. Para un camino elemental la cláusula es:

camino(X,Y,1) :- arco(X,Y).

2. Para un camino no elemental entre X e Y, L1 será la longitud entre Z-Y. Si hay un arco de X a Z la longitud del camino X a Y es: $L = L1 + 1$

El programa queda:

camino(X,Y,1) :- arco(X,Y).

camino(X,Y,L) :- arco(X,Z), camino(Z,Y,L1), L is L1 + 1.

L es calculada luego de la llamada recursiva, porque L1 debe ser instanciada antes de la adición.

En este programa hay una operación que debe realizar luego de la llamada recursiva camino. Esta operación se debe ejecutar en el orden reverso que las llamadas recursivas por que es ejecutada después de camino. La longitud del primer camino es conocida solo luego que las llamadas recursivas han finalizado.

camino(c,d,1).

Luego que camino(c,d,1) es probado PROLOG ejecuta $L \text{ is } L1 + 1$ de la segunda llamada. A este nivel el valor de L1 se conoce: $L1=1$.

camino(b,d,L):- arco(b,c),camino(c,d,1),L is 1+1.

En el nivel 2, $L=2$, hay dos arcos entre b y d. Finalmente y de la misma manera:

camino(a,d,L):- arco(a,b),camino(c,d,2),L is 2+1.

A este nivel, $L=3$

?- camino(a,d,L).

$L=3$.

PROLOG maneja la recursión con un mecanismo clásico de pilas. Por lo tanto podría producirse overflow (luego de cientos de llamadas) Por lo tanto, para un problema complejo que tiene el riesgo de producir muchísimas llamadas, es mejor descomponer el problema en varias partes. La pila es vaciada luego que finaliza cada serie de llamadas recursivas.

ESTRUCTURAS Y MANEJO DE CONTROL

Los mecanismos que gobiernan la resolución de un problema son llamado mecanismo de control de resolución.

Control de estructuras

Durante la ejecución de una consulta, PROLOG maneja llamadas a cláusulas, unificación y backtracking.

Decimos que el sistema de PROLOG controla la ejecución de la consulta o la demostración del objetivo.

Si detenemos la ejecución en un determinado instante, las diferentes tareas pueden clasificarse en una de las 3 siguientes categorías:

- Los objetivos probados
- Los objetivos que están en proceso de prueba
- Los objetivos que esperan ser probados

Además, cada objetivo que ya ha sido probado tienen asociado con el, en el árbol de prueba, una lista de cláusulas en el procedimiento que no han sido consideradas aun por la ejecución en progreso. Llamamos a esta lista de “lista de elecciones pendientes”. Esta lista contiene el número de cláusula que aun no ha sido usadas para probar la demostración del procedimiento que se esta considerando.

Por ejemplo el siguiente programa:

(1) a :- c,d.

(2) a :- a.

(1) c :- d,a.

(2) c :- e.

(3) c :- e,c.

(1) e :- d,f.

(2) e :- .

(1) f :- a.

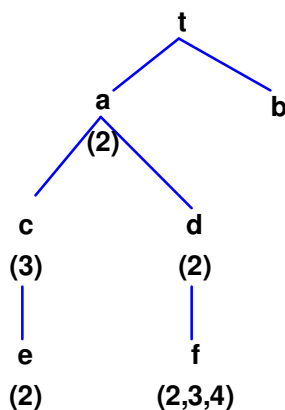
(2) f :- c.

(3) f :- d,e,f.

(4) f :- a,d.

Llamada: t: -a,b.

Observemos en un punto arbitrario en el desarrollo de la prueba, se representa el siguiente árbol:



Para el objetivo a, la elección 2, no ha sido considerada. Para el objetivo f, que esta en procedimiento de ser probado, la lista de elecciones pendientes es 2,3,4. Esto significa que las cláusulas 2,3 y 4 del procedimiento f no han sido aun ejecutadas para probar f, en este preciso punto del árbol de prueba. Las cláusulas 2,3 y 4 sin elecciones que no han sido consideradas por PROLOG pero lo serán mas adelante.

Cada llamada a un procedimiento tiene su propia e independiente lista de elecciones pendientes. Si un procedimiento es llamado varias veces es claro que la lista de elecciones asociada a cada llamada es independiente de las otras.

En el ejemplo:

- Objetivos c y e han sido probados
- Objetivos a, d y f están en proceso de ser probados
- Objetivo b esta pendiente de prueba.

Proceso de ejecución

PROLOG se caracteriza por realizar una estrategia de construcción de la prueba. Esta interpretación ha sido principalmente en términos de eficiencia de memoria.

Dos procesos afectan el progreso de la ejecución de un programa:

- (1) El progreso normal para probar objetivo, las llamadas son ejecutadas de un modo secuencial en el orden en que están escritas en el programa.
- (2) Backtracking, ocurre luego que se ha hallado una solución en un problema (fracaso-éxito)

En cada llamada, las variables introducidas por dicha llamada son substituidas por otras variables o por términos funcionales. Se nota que:

- (1) Dos variables inicialmente libres y diferentes, pueden ser ligadas juntas durante la substitución
- (2) Una vez substituido un término, una variable no puede ser substituida por otro término, excepto cuando la variable es liberada del valor durante el backtracking.

Si una variable es substituida por un término que contiene variables libres, estas nuevas variables deben, a su turno, ser afectados por substituciones.

Cuando el backtracking ocurre, esto significa que no hay posibles elecciones a tal punto de la prueba, la lista de elecciones pendientes esta vacía. El sistema entonces retorna a la precedente elección y restaura el estado de las variables a ese nivel. La próxima cláusula dada por la lista de elecciones para esa instancia de la llamada es entonces considerada. Si la lista esta vacía, el sistema sube a otro nivel.

La estrategia de análisis de PROLOG por razones de eficiencia y economía de espacio de memorias es la clásica estrategia de “left to right” o “depth-first search” (búsqueda en profundidad)

Para ilustrar el mecanismo de resolución analizamos el siguiente ejemplo:

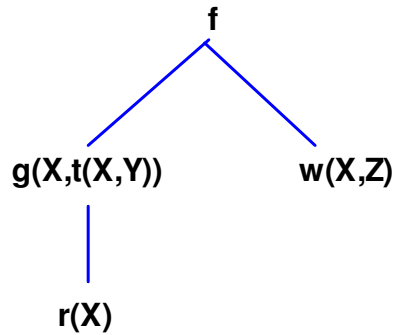
Programa

f:- g(X,Y), w(X,Z).

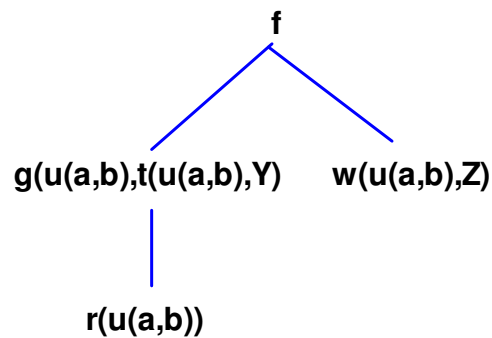
g(X,t(X,Y)) :- r(X).

r(u(a,b)).

Una vez que g es llamada, la llamada de f produce:



Porque Y es reemplazada por t(X,Y) durante la unificación. La variable Y en t(X,Y) no es la misma que Y en la llamada a g de la cláusula f. Luego de la ejecución de r, obtenemos:



Usando literales sin argumentos por simplicidad, ilustramos con un ejemplo corto, la estrategia de resolución. Asumimos las siguientes cláusulas:

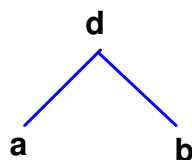
a :- f.
a :- g.
b :- h.
b :- q.

Y los hechos:

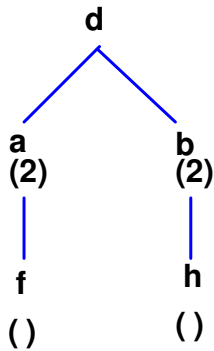
f.
g.
h.
q.

La llamada a probar es: d :- a,b.

El árbol inicial de resolución es:



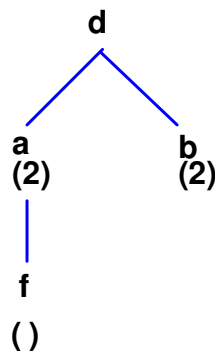
Para probar a intentamos usar la regla a :- f. ; para probar b intentamos probar a regla b :- h.



La lista de elecciones esta vacía para f y h porque son hechos y por lo tanto no tienen alternativas

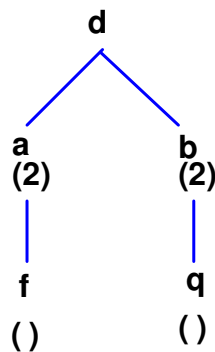
➤ Esto constituye la primer prueba.

Otro caso se presenta si se retorna b (si h no ha fallado)

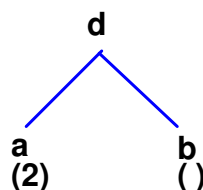


Se trata de probar b :- q.

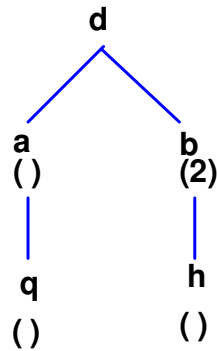
No hay más alternativas después de q. Esta es una segunda posible prueba. Si hay un retorno a b(fracaso o existo) entonces no hay mas caminos para probar b y se retorna a a.



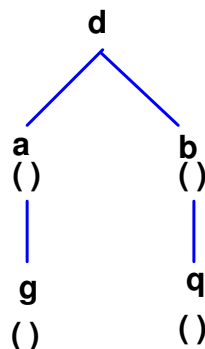
Es posible que otra prueba de d pueda tener consecuencias que no permiten probar b, cuando no hay argumentos.



Tratamos nuevamente de probar a, su segunda posible elección, es la regla a :- g. Y b con la regla b :- h. Obtenemos la tercer prueba posible:



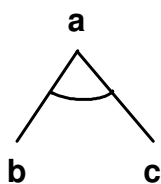
Retornamos entonces a b y todavía hay elecciones pendientes:



Esta es la cuarta y última prueba. Si falla en la 4ta. elección entonces d no puede ser probado.

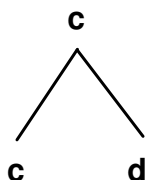
Este conjunto de pruebas puede representarse de una manera más concisa, mediante los significados los árboles de AND-OR. Este tipo de árboles permiten una representación de conjuntos elementales que para toda rama deben ser probados (AND nodos) y el grupo de elementos en los cuales solamente uno debe ser probado para asegurarse la verdad del resultado (OR nodos).

AND nodos son representados de la siguiente manera:



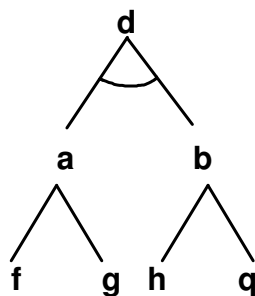
Significa que para probar a es necesario probar b y c.

OR nodos:



Significa que C es verdad si d o e es verdad.

En un árbol AND-OR, los nodos AND y OR aparecen alternadamente.



Este árbol significa que para probar *d*, se deben probar *a* y *b*. Para probar *a* se debe probar *f* o *g*, y para probar *b* se debe probar *h* o *q*.

Este tipo de representación es muy conveniente para ver claramente los distintos posibles caminos en la ejecución de la prueba. El sistema de PROLOG examina las diferentes ramas en orden primero en profundidad, comenzando con la rama mas a la izquierda.

Predicados para controlar la resolución

Hay dos predicados predefinidos que permiten al programador controlar el proceso de prueba:

- (1) el predicado que provoca que falle la prueba
- (2) el predicado que permite la supresión de elecciones (alternativas)

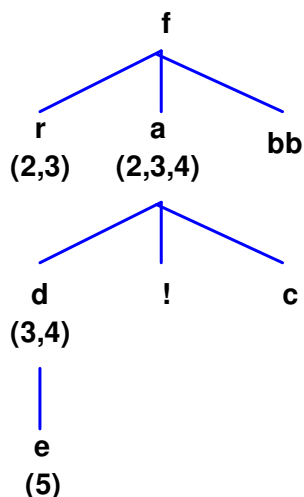
El predicado “**fail**” siempre que se evalúa es verdadero, y cuando se ejecuta el valor de la actual cláusula se vuelve falso.

a :- b,c,fail.

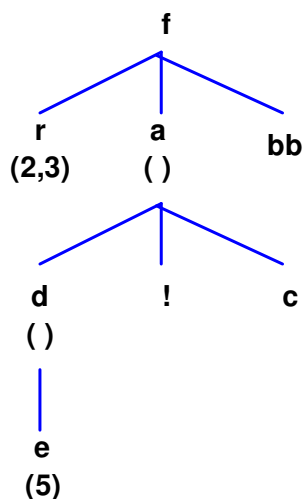
Aquí si *a* :- *b,c* es evaluado verdadero, la ejecución de *fail* hace a falso. Si *b* o *c* son falsos, *fail* nunca es ejecutado, *a* es también falso, pero por diferentes razones.

El predicado predefinido que suprime las alternativas pendientes (llamado **cut**) se escribe **!** Este predicado siempre que se es evaluado es verdadero y cuando es ejecutado, suprime todas las posibles alternativas para todos los literales en la cabeza de la actual cláusula, incluyendo la cabeza del predicado.

Ejemplo



Luego de la ejecución de ! el estado es:



El control es inmediatamente pasado a c, y todos los backtracking desde c van directamente a f. Todas las alternativas pendientes a a y d han sido eliminadas por cut. Por lo tanto la alternativa pendiente r no será considerada. Ahora se considerarán las alternativas pendientes en r.

- El símbolo cut, esencialmente permite escribir programas eficientes, evitando la necesidad de buscar todas las posibilidades cuando queremos conocer solamente si existe al menos una solución para un problema.
- También permite una más conveniente expresión de if A then B else C, ya que la construcción no requiere que el condicional se escriba dos veces. En este caso la parte then contiene el cut.
- El cut debe ser usado con extremo cuidado, porque puede cambiar el significado del programa. El uso de cut usualmente hace que el orden de las cláusulas muy importante. A pesar de esta restricción el uso de cut tiene sus ventajas.

Ejemplos 1

Hechos

```

pais(francia,paris,frances).
pais(espania,madrid,espaniol).
pais(italia,roma,italiano).
  
```

Reglas

$\text{capital}(X) \text{ :- pais}(_,X,_).$ O $\text{capital}(X) \text{ :- pais}(_,X,_), !.$

En la primer opción de la regla capital, luego que encuentra la capital sigue examinando la base de hechos, aunque ya sabemos que una ciudad no puede ser capital de dos países. En la segunda opción de la regla, luego que encuentra el país, cuya capital es X, finaliza la búsqueda.

Ejemplos 2

Regla

$\text{padre}(X,Y) \text{ :- hijo}(Y,X).$ O $\text{padre}(X,Y) \text{ :- hijo}(Y,X), !.$

En la primer opción, luego que encuentra un hijo continúa buscando, en la segunda luego que encuentra un resultado verdadero se detiene.

Ejemplos 3

Regla

$\text{max}(X,Y,X) \text{ :- } X \geq Y.$

$\text{max}(X,Y,Y) \text{ :- } Y > X.$

O

$\text{max}(X,Y,X) \text{ :- } X \geq Y, !.$

$\text{max}(X,Y,Y).$

Consideremos el predicado $\text{diff}(X,Y)$ que retorna verdadero si X e Y no son unificables. El programa es:

$\text{diff}(X,X) \text{ :- } !, \text{fail}.$

$\text{diff}(X,Y).$

Si hay unificación con la cabeza de la primer cláusula, el cut tiene efecto y la segunda cláusula $\text{diff}(X,Y)$ nunca será ejecutada a este nivel de profundidad de la prueba. En la primer cláusula el uso de la misma variable X en los dos argumentos fuerza a que sean iguales y por lo tanto no diferentes. La primer cláusula termina con fail en orden que si la unificación sucede, el resultado será falso. La segunda cláusula es activada solamente si la primera no puede ser usada, es decir, si los dos argumentos de diff no pueden ser unificados con la misma constante.

?- $\text{diff}(a,a).$ Resultado falso

?- $\text{diff}(a,b).$ Resultado verdadero

Como se ve en este ejemplo el cut se usa para hacer programas mas eficientes o para cambiar el significado del programa.

➤ Tipos de Cut

Red Cut : debe ser usado con cuidado, cambia el significado

Green Cut: mejora la eficiencia.

Negación por Falla

El predicado $\text{not}(P)$ retorna verdadero si P es falso y falso si P es verdadero. El programa es:

$\text{not}(P) \text{ :- } P, !, \text{fail}.$

$\text{not}(P).$

- Si la unificación con la cabeza de la primer cláusula tiene éxito, PROLOG intenta ejecutar P. P siempre puede ser instanciado con un literal en la ejecución. Si el resultado de probar P es verdadero, el cut es involucrado. La segunda cláusula no será ejecutada y el predicado fail hace

que PROLOG retorne falso. Si P es falso, la primer cláusula retorna falso, la segunda cláusula es ejecutada y retorna verdadero.

- El sistema logra proceder con la segunda cláusula cuando no alcanza el cut de la primer cláusula.
- El programa significa que si P es probado el resultado es falso sino el resultado es verdadero (no es posible probar P).

En algunas versiones de PROLOG las variables pueden aparecer solamente como argumentos. Aca P es un término y constituye una condición a ser verificada. Por esa razón **not** es por lo general un predicado predefinido.

El uso de la negación incrementa el poder expresivo de PROLOG al autorizar la negación de condiciones. Ahora podemos:

diff(X,Y) :- not(X = Y).

Para la relación hermano(X,Y), que es verdadera si X es hermano de Y se puede definir no_hermano(X,Y), que será verdadero si X no es hermano de Y :

no_hermano(X,Y) :- not hermano(X,Y).

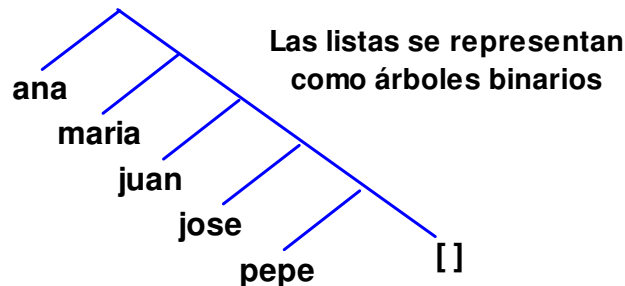
no_hermano(X,Y) da falso, significa que no hay suficiente información en la base de hechos para probar la consulta. Todo lo que no puede probar se asume como falso.

Lo ideal sería tener tres posibles respuestas: verdadero, falso, no se.

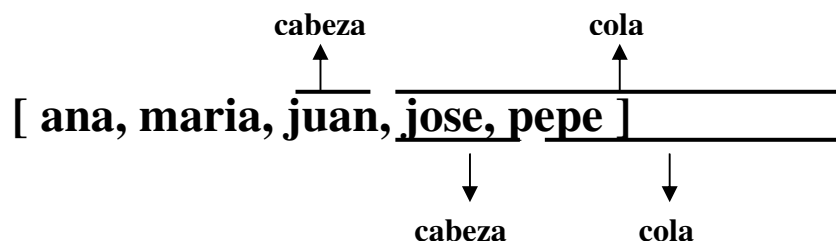
LISTAS

- Las listas son la estructura de datos mas utilizada en PROLOG.
- Las listas permiten ensamblar conjuntos de objetos de distintos tipos.

$E = \{ \text{ana, maria, juan, jose, pepe} \}$ // el conjunto E y sus elementos
 $[\text{ana,maria,juan,jose,pepe}]$ // representación de E en PROLOG



En una lista los elementos estan “ordenados”. El primer elemento se llama “cabeza” de la lista y los elementos subsecuentes “cola” de la lista.



La cola es una sublista que también tiene cabeza y cola, y así recursivamente. Hasta la sublista pepe, pepe es la cabeza y la lista [] (lista vacía) es la cola [pepe,[]].

Cada elemento de una lista puede ser una lista o un término. Ejemplo $[a(X,Y),[b,c,d,e],Z]$ esta lista contiene dos términos y una lista.

- La estructura de datos lista permite representar grupos de objetos diversos
- A priori, no existe relación entre los elementos.

Unificación

La operación fundamental es la separación de la cabeza de la cola. Esto requiere dos variables, X para designar la cabeza e Y para designar la cola. El símbolo | se usa para separar la cabeza de la cola.

Tenemos la lista $[a,b,c,d,e]$. Es unificado con $[X|Y]$. X e Y son ligadas como sigue:

$X = a$

$Y = [b,c,d,e]$

En el caso que la lista tenga un solo elemento $[a]$, $[X|Y]$ es:

$X = a$

$Y = []$

- La unificación con una lista vacía, [], falla, porque una lista vacía no puede ser dividida en dos elementos.

Problema: Búsqueda de un elemento en una lista.

Reglas

miembro(X, [X|Y]).

miembro(X, [Y|Z]) :- miembro(X,Z).

Consultas

?- miembro(4,[4,5,6]).

yes

?- miembro(3,[5,3,6]).

yes

?- miembro(2,[4,5,6]).

no

?- miembro(2,[4,2,5,2,3,2]).

yes // encuentra el 2 tres veces

Veamos las reglas:

miembro(X, [X|Y]). // el elemnto es miembro si es igual a la cabeza de la lista (X)

miembro(X, [Y|Z]) :- miembro(X,Z). // llama a miembro cuando el X es diferente a la cabeza de la lista (Y), enviando la cola(Z).

Es decir, se llama recursivamente enviando como lista la cola (sublista) hasta que envia la lista [] que no unifica con ninguna de las dos cláusulas y falla.

Otras operaciones

➤ Subconjunto

subconjunto(X,Y). X e Y son dos listas. Subconjunto(X,Y) retorna verdadero si todos los elementos de X están en Y.

?- subconjunto([3,4],[5,3,1,4,6]).

Yes

Se debe comprobar que cada elemento de X es miembro de Y. Se detiene cuando X no tiene mas elementos.

Procedimiento

subconjunto([],Y).

subconjunto([X1|X2],Y) :- miembro(X1,Y),subconjunto(X2,Y).

➤ Cantidad de elementos

cant([],0).

cant([X1|X2],C) :- cant(X2,C1), C1 is C1 + 1.

➤ Concatenar dos listas

con(L1,L2,L3) concatena L2 a L1 en L3.

Si L1 = [a,b,c] y L2 = [d,e,f] entonces L3 = [a,b,c,d,e,f]

Consulta:

?-con([a,b,c], [d,e,f],L).

L=[a,b,c,d,e,f]

Procedimiento

con([],Y,Y). // el resultado de concatenar la lista vacía a Y es Y.
con([X|Y],W, [X|T]) :- con(Y,W,T).

Veamos:

1. Unifica la cabeza de L1 con la cabeza de L3 (agrega el primer elemento).
2. Llama a con con la cola de L1 y L3
3. L2 se concatena toda de una vez con la primer cláusula cuando L1 = [].
4. T Toma valor en el último paso.

L1	L2	L3
[a,b,c]	[d,e,f]	[]
[b,c]	[d,e,f]	[a]
[c]	[d,e,f]	[a,b]
[]	[d,e,f]	[a,b,c]
[]	[d,e,f]	[a,b,c,d,e,f]

El programa con puede ser usado para otros propósitos:

?-con(X,[d,e,f], [a,b,c,d,e,f]).
X=[a,b,c]

?-con([a,b,c],Y, [a,b,c,d,e,f]).
X=[d,e,f]

?-con(X,Y, [a,b,c,d]).
X=[] Y=[a,b,c,d];
X=[a] Y=[b,c,d];
X=[a,b] Y=[c,d];
X=[a,b,c] Y=[d];
X=[a,b,c,d] Y=[];

- **Hay un solo método para acceder a los elementos de una lista en PROLOG: extrayendo la cabeza de la cola de la lista. Este método siempre empieza por el principio de la lista.**